当前 SDK 中存在连续两次 build，生成的 system 差异较大，也就导致同一个 SDK 版本即便不改代码，编出来的两个版本做出来的 FBF 差分包也会异常很大！

问题原因：经过上图我们也知道，FBF 差分包制作的大是因为 system.img 间差异很大，而 system 在编译前后差异过大的原因是 feeaback 机制导致，feedback 在每次编译时会对冗余代码进行优化，链接结束后又会生成新的优化的 feedback（优化掉实际链接不用的接口），而下一次编译时使用的 feedback 是经过上一次优化后的，这就导致了两次编译实际上使用的不是一个 feedback，所以两次编译的优化代码部分会有差距，也就是为什么 system 的差异过大的原因了；

解决方法：在 SDK 实际编译前，我们先保存本次编译实际使用的 feedback 到固件包中的 DBG 目录下，客户如果是在同一个 SDK 上进行 fbf 方式 fota 升级（且实际几乎不改动 kernel 侧代码），就可以将基础版本（旧版本）DBG 中的 feedbackLink_backup.txt 取出，然后替换掉 ql-kernel\threadx\config\common 下的 feedbackLinkOptLteonlyThin.txt 文件，重新编译后生成的 system.img 同基础版本（旧版本）的 system 差异就会很小了（每次替换编译只有本次替换有效，后续编译还需要继续），这样做出来的差分包也就在 50K 以下的水平（前提是没有改动 kernel，改动后按照改动影响决定差分包大小）

需要注意的是，替换 feedback 的方法也有一个明显的弊端：如果新版本较旧版本有裁剪功能，但是编译时使用的还是旧版本的 feedback，那么被裁剪的功能就不一定能释放出空间，也就是 system.img 中还是会存在此功能的代码并占用空间，所以实际操作时需要根据实际情况自行评估是否需要使用此方式减小 system 间的差异。

首次量产前的版本无需关注 feedback 优化，以实际生成的固件包使用即可，但首次固件量产的时候需要注意保存固件包中 DBG 目录下的 feedbackLink_backup.txt，因为如果在固件量产后续需要进行 FBF 方式 fota 升级内核场景时，当差分包过大（也就是两版本间 system 差异过大）可能无法使用时，可以按照上述方法将 feedback 文件替换后编译生成固件，这样可以有效降低 system 间的差异，即减小差分包的大小。